SQL

# Interviewer: "Your JOIN doubled the row count. Why?"

## Comment your answer

The full answer: why it happens, how to prove it, 3 ways to fix it, and the follow-up questions interviewers ask next.

INSIDE THIS PDF · 8 PAGES

- The short answer
- See it happen
- Prove it in 3 queries
- Fix 1: aggregate first, then join
- Fix 2 and Fix 3
- Follow-up questions
- How to answer in the interview

@thedataguy16

# The short answer

Say this first, in one breath. Then show you can prove it.

- **The join key is not unique on one side**

  One row on the left matches more than one row on the right, so the left row is repeated once for every match.

- **It is not a bug in SQL**

  JOIN returns every matching pair. The data has duplicates, or you joined at the wrong grain.

- **The real danger**

  Row count is the visible symptom. The silent damage is inflated SUM and COUNT in every report built on top.

**Grain = what one row means. orders: one row per order. order_items: one row per item. Joining them makes one row per item.**

@thedataguy16

# See it happen

One customer, two address rows. Every order for that customer now appears twice.

```
customers
id | name
1  | Asha

addresses
customer_id | city
1           | Pune
1           | Mumbai   <- duplicate key

orders
order_id | customer_id | amount
101      | 1           | 500
102      | 1           | 300

SELECT o.order_id, o.amount, a.city
FROM orders o
JOIN addresses a
  ON a.customer_id = o.customer_id;

order_id | amount | city
101      | 500    | Pune
101      | 500    | Mumbai
102      | 300    | Pune
102      | 300    | Mumbai

2 orders became 4 rows.
SUM(amount) = 1600. Real total = 800.
```

@thedataguy16

# Prove it in 3 queries

Interviewers want to see how you debug, not just the definition.

## 1. Count before and after

If the joined count is higher than the left table count, rows are multiplying.

```
SELECT COUNT(*) FROM orders;                    -- 2
SELECT COUNT(*) FROM orders o
JOIN addresses a USING (customer_id);           -- 4
```

## 2. Find the duplicate keys

Any key with COUNT above 1 on the right side is the cause.

```
SELECT customer_id, COUNT(*)
FROM addresses
GROUP BY customer_id
HAVING COUNT(*) > 1;
```

## 3. Check the grain

Ask: what does one row of each table mean? If they differ, the join must handle it.

@thedataguy16

# Fix 1: aggregate first, then join

Best when the right table has many rows per key and you only need a total from it.

```
WITH item_totals AS (
  SELECT order_id,
         SUM(qty)         AS total_qty,
         SUM(qty*price)   AS item_value
  FROM order_items
  GROUP BY order_id              -- now one row per
order
)
SELECT o.order_id, o.order_date, t.total_qty,
t.item_value
FROM orders o
LEFT JOIN item_totals t ON t.order_id =
o.order_id;
```

## Why it works

Both sides are now at the same grain: one row per order.

@thedataguy16

# Fix 2 and Fix 3

When you need one specific row, or just want to know if a match exists.

## Fix 2: pick one row with ROW_NUMBER

Keep the latest address per customer, then join.

```
WITH a AS (
  SELECT *, ROW_NUMBER() OVER (
    PARTITION BY customer_id
    ORDER BY updated_at DESC) AS rn
  FROM addresses)
SELECT o.*, a.city
FROM orders o
JOIN a ON a.customer_id = o.customer_id
     AND a.rn = 1;
```

## Fix 3: EXISTS when you only filter

You want orders from customers who have any Mumbai address. No rows are added.

```
SELECT o.* FROM orders o
WHERE EXISTS (
  SELECT 1 FROM addresses a
  WHERE a.customer_id = o.customer_id
    AND a.city = 'Mumbai');
```

**Do not fix it with DISTINCT. It hides the symptom, is slow on big tables, and SUM can still be wrong.**



@thedataguy16

# Follow-up questions

Interviewers usually ask one of these next. Have the answer ready.

- **"Would LEFT JOIN fix it?"**

  No. LEFT JOIN keeps unmatched rows. Duplicate keys multiply rows in any join type.

- **"How do you stop this in production?"**

  Add a unique constraint or a data test on the key (for example dbt unique test). Check row counts after every join in a pipeline.

- **"When is a row increase correct?"**

  When you meant to change the grain, like orders to order items. Then name the output clearly and never sum order-level columns.

- **"Why not just use DISTINCT?"**

  It hides the problem and two genuinely different rows with the same values collapse into one.

@thedataguy16

# How to answer in the interview

A 40-second answer that sounds experienced.

## 1. Cause

"The join key is not unique on the right table, so each left row is repeated for every match."

## 2. Proof

"I would compare row counts before and after, then GROUP BY the key with HAVING COUNT(*) > 1."

## 3. Fix

"Depending on the need: aggregate the right table first, pick one row with ROW_NUMBER, or use EXISTS if I only need to filter."

## 4. Prevention

"And I would add a uniqueness test on that key so it does not happen again."

**Comment test: if your answer said "duplicate keys", you got the core right. The proof and the fix are what get you hired.**

@thedataguy16